

Robot Movement Planning

Constraint Programming Project Report

Alessio Russo (mat. 376856)

1 Problem Statement

We consider a stream of identical boxes arriving on a belt. Each box has size $x \times y$, where x is parallel to the belt direction and y is orthogonal to it. A target pallet capable of containing n boxes is given. Each box i has a destination position on the pallet, given by $(P_x[i], P_y[i])$, where $i = 1, \dots, n$.

The robot can pick a consecutive set of boxes from the stream. Since boxes on the belt have no gaps between them, each picked set is contiguous along the x -axis. Given a gripper size S , the total length of the picked set must be at most $S + x$. This corresponds to an overflow of at most $x/2$ on each side of the gripper. Therefore, the maximum number of boxes that can be picked at once is $\left\lfloor \frac{S+x}{x} \right\rfloor$.

After picking a set of boxes, the robot releases it on the pallet. Figure 1 shows an example of the picking and releasing operations.

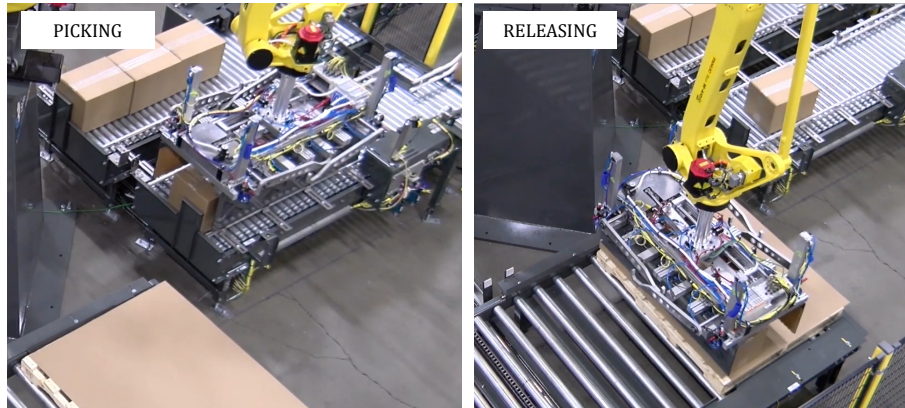


Figure 1: Example of picking and releasing operations.

The gripper has two parallel plates, but only one plate can move during release. For this reason, the opening side must be free. It can face either the outside of the pallet or a position that has not been filled yet. We split the problem into two parts:

1. **Grouping.** Identify the sets of boxes that can be picked together. Each group must contain boxes on the same row, aligned along the x -axis, and must fit within the gripper capacity.
2. **Scheduling.** Choose the order in which the groups are released. For each group, also choose which plate opens. At every step, the selected opening side must be free.

The goal is to find a sequence of pick-and-release actions that places all boxes in their target positions while respecting the gripper capacity and avoiding collisions during release. The code used for the model and the benchmark instances is available at [...](#)

1.1 Parameters and Derived Constants

The proposed MiniZinc model uses five input parameters. The parameter `n_boxes` denotes the total number of boxes in the layer. The parameters `boxes_along_x` and `boxes_along_y` define the grid dimensions of the layer. The parameter `boxes_x_dim` denotes the box dimension parallel to the gripper. In the natural orientation, this corresponds to x ; if the layer is rotated, it corresponds to y . Finally, `grip_size` denotes the gripper size S . These parameters are declared as follows:

```
1 par int: n_boxes;
2 par int: boxes_x_dim;
3 par int: boxes_along_x;
4 par int: boxes_along_y;
5 par int: grip_size;
```

From these inputs, the model derives three constants. The first one is the maximum number of boxes that can be picked in a single operation, computed as:

$$\text{max_pkble_boxes} = \left\lceil \frac{\text{grip_size} + \text{boxes_x_dim}}{\text{boxes_x_dim}} \right\rceil.$$

This value derives directly from the gripper capacity $S + x$. The second constant is the number of groups needed to cover one pallet row:

$$\text{groups_per_row} = \left\lceil \frac{\text{boxes_along_x}}{\text{max_pkble_boxes}} \right\rceil.$$

This is a ceiling division because the last group in a row may contain fewer boxes than the maximum allowed number. The third constant is the total number of groups:

$$\text{n_groups} = \text{boxes_along_y} \cdot \text{groups_per_row}.$$

It is obtained by multiplying the number of groups per row by the number of rows. The corresponding MiniZinc declarations are:

```
1 par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;
2
3 par int: groups_per_row = (boxes_along_x + max_pkble_boxes - 1)
4                       div max_pkble_boxes;
5
6 par int: n_groups = boxes_along_y * groups_per_row;
```

2 Grouping

This part of the model admits several possible encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each group explicitly stores the identifiers of all boxes it contains. Since groups may have different lengths, this representation requires a two-dimensional array with padding values for shorter groups.

```
1 array[1..n_groups, 1..max_pkble_packs] of var 0..n_packs: groups;
```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes group membership explicit and easy to inspect. However, it introduces a large number of decision

variables and requires additional constraints to enforce the structure of a valid solution. In particular, the model must ensure that each box appears exactly once, that non-empty positions inside each group are ordered, and that the groups are consistent with the geometric ordering of the layer.

A second possibility is a *binary assignment* encoding, where a boolean variable states whether pack i belongs to group g . This representation can be written as follows:

```
1 array[1..n_groups, 1..n_packs] of var bool: belongs_to_group;
```

In this case, group membership is represented by the truth value of `belongs_to_group[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation. It also does not encode contiguity or same-row membership directly. These properties must be reconstructed through additional constraints.

For this reason, the adopted model represents each group using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each group is described by only two main variables. Contiguity is implicit: once the start index and the length are known, the boxes in the group are uniquely determined. The same-row constraint can also be expressed directly by requiring the interval not to cross a row boundary. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid groups.

```
1 array[1..n_groups] of var 1..n_packs: group_start;
2 array[1..n_groups] of var 1..max_pkble_packs: group_len;
```

The adopted encoding uses two arrays: `group_start`, which stores the first box of each group, and `group_len`, which stores the number of boxes in that group.

```
1 constraint group_start[1] = 1;
2
3 constraint forall(g in 1..n_groups - 1)(
4   group_start[g + 1] = group_start[g] + group_len[g]
5 );
6
7 constraint group_start[n_groups] + group_len[n_groups] - 1 = n_packs;
8
9 constraint forall(g in 1..n_groups)(
10   (group_start[g] - 1) div packs_along_x =
11   (group_start[g] + group_len[g] - 2) div packs_along_x
12 );
```

The first three constraints define the groups as consecutive intervals. The first group starts from pack 1, and each following group starts immediately after the last pack of the previous group. In particular, the third constraint forces the last group to end at pack n . Together, these conditions ensure that all packs are covered exactly once, without gaps or overlaps.

The fourth constraint enforces the geometric feasibility of each group: a group cannot cross from one row to the next. The constraint compares the row index of the first pack with the row index of the last pack in the group. If the two indices are equal, the whole interval lies on the same row.

An example of the result of the grouping phase is shown in Figure 2.

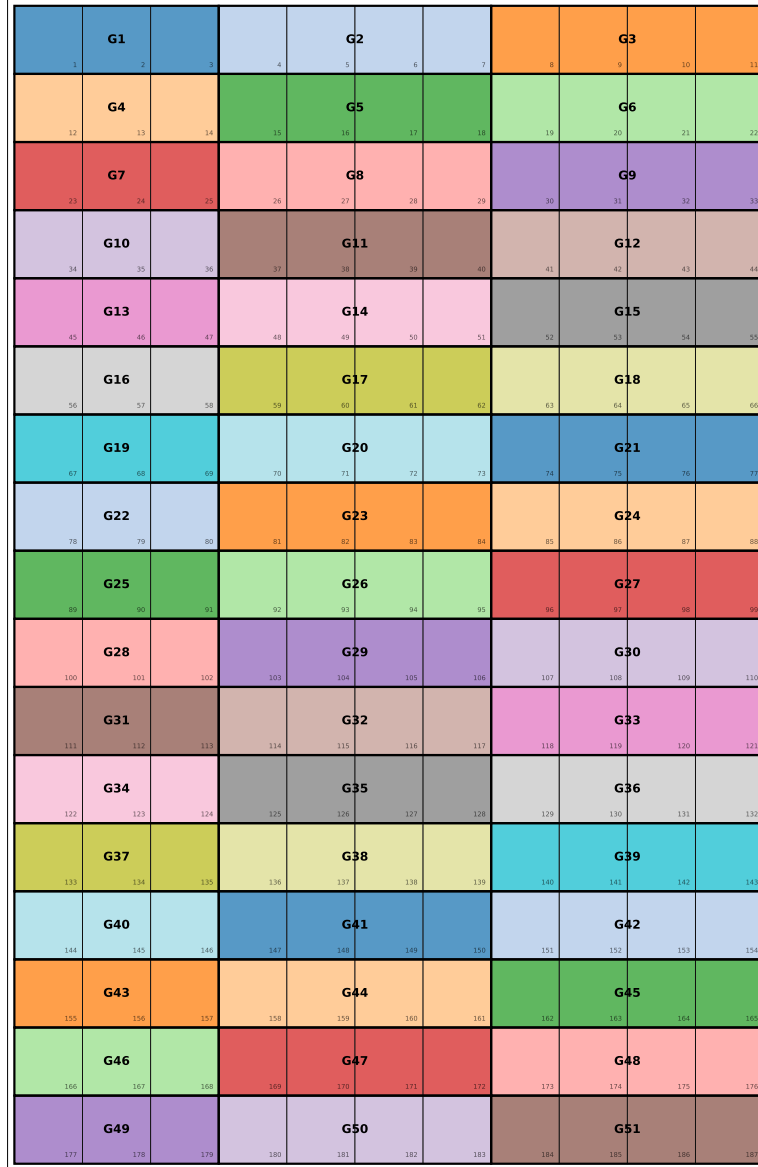


Figure 2: Pallet layout produced by the grouping stage: 51 groups tiling the 11×17 grid, each colored distinctly. Groups alternate 3–4–4 across each row.

3 Scheduling

The second part of the model determines the release order of the groups and selects which gripper plate opens for each release. In this case, the most direct representation consists in assigning to each group a single release position. The array `drop_order` stores the time step at which each group is released, while `plate` stores the plate opened for that release. The value 0 denotes the left plate, and the value 1 denotes the right plate.

```
1 array[1..n_groups] of var 1..n_groups: drop_order;
2 array[1..n_groups] of var 0..1: plate;
```

The only global constraint required to make `drop_order` a valid schedule is `alldifferent`. Since every value in `drop_order` belongs to the domain $1, \dots, n_groups$, forcing all values to be

different ensures that each release step is assigned to exactly one group.

```
1 constraint alldifferent(drop_order);
```

The feasibility of a release depends on the position of the adjacent groups in the same vertical column. As shown in Figure 2, groups are numbered from left to right within each row, and then from top to bottom across rows. Therefore, since each row contains `groups_per_row` groups, two groups that are aligned in consecutive rows have indices that differ by `groups_per_row`. For a generic group g , the vertical neighborhood is therefore given by $g \pm \text{groups_per_row}$, whenever these indices are within the valid range. Groups in the first pallet row have no upper neighbor, while groups in the last pallet row have no lower neighbor.

The final constraint encodes the physical restriction on the gripper plates: when a group is released, the selected plate must open toward a free direction. A direction is free if the group lies on the corresponding pallet boundary, or if the group in that direction is scheduled to be released later and is therefore still absent from the pallet at the current release step.

Since the relevant neighbors are above and below the current group, the constraint is written by separating the first row, the internal rows, and the last row. This avoids invalid array accesses at the pallet boundaries.

```
1 constraint forall(g in 1..n_groups where g <= groups_per_row)(
2   plate[g] = 0 /\
3   (
4     plate[g] = 1 /\
5     drop_order[g + groups_per_row] > drop_order[g]
6   )
7 );
8
9 constraint forall(g in 1..n_groups where
10  g > groups_per_row /\ g <= n_groups - groups_per_row
11 )(
12   (
13     plate[g] = 0 /\
14     drop_order[g - groups_per_row] > drop_order[g]
15   )
16   /\
17   (
18     plate[g] = 1 /\
19     drop_order[g + groups_per_row] > drop_order[g]
20   )
21 );
22
23 constraint forall(g in 1..n_groups where g > n_groups - groups_per_row)(
24   (
25     plate[g] = 0 /\
26     drop_order[g - groups_per_row] > drop_order[g]
27   )
28   /\
29   plate[g] = 1
30 );
```

The first constraint handles groups in the first pallet row. These groups have no upper neighbor, so opening plate 0 is always feasible. Opening plate 1, instead, is feasible only if the lower neighbor is released later.

The second constraint handles the internal rows. In this case, both vertical neighbors exist. Opening plate 0 is feasible only if the upper neighbor $g - \text{groups_per_row}$ is released later.

Similarly, opening plate 1 is feasible only if the lower neighbor $g + \text{groups_per_row}$ is released later.

The third constraint handles groups in the last pallet row. These groups have no lower neighbor, so opening plate 1 is always feasible. Opening plate 0 is feasible only if the upper neighbor is released later.

The model is formulated as a feasibility problem:

```
1 solve satisfy;
```

No objective function is required. Any assignment of `drop_order` and `plate` that satisfies the vertical-neighbor constraint defines a valid pick-and-place plan. Figure 3 shows an example of such a schedule at different placement steps.

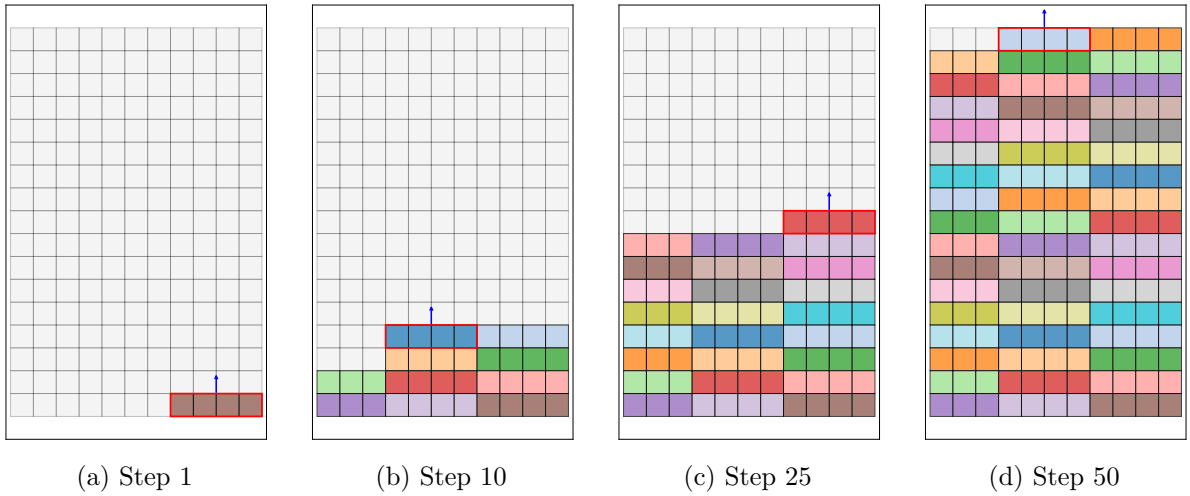


Figure 3: Example of a feasible scheduling sequence. The red border marks the group being placed at the selected step, while the blue arrow indicates the opening side of the plate.

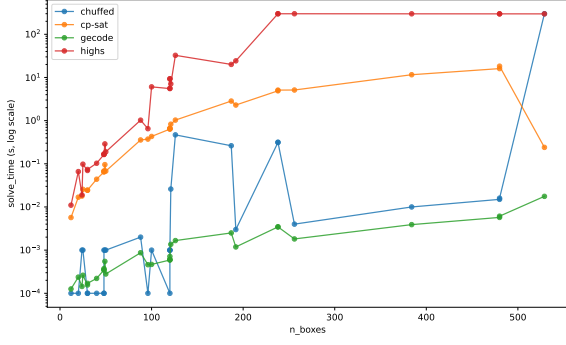
4 Calling the Model from Python

Small Python scripts are used to interface with the MiniZinc model. The model is loaded through `minizinc-python` and associated with the Gecode solver:

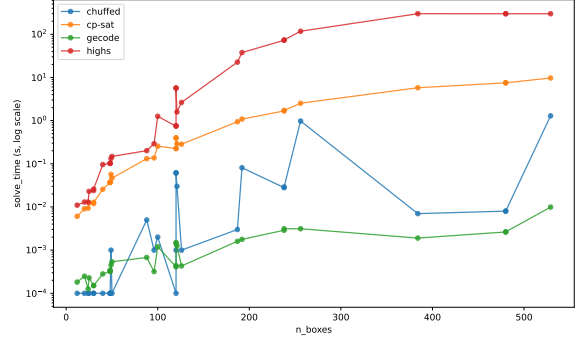
```
1 model      = Model("model.mzn")
2 solver     = Solver.lookup("gecode")
3 instance   = Instance(solver, model)
```

The scripts then assign concrete values to the MiniZinc parameters and start the solving process:

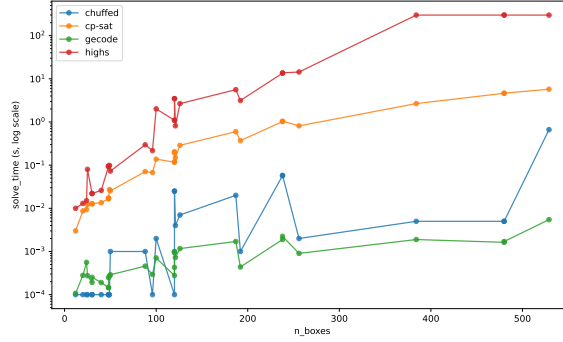
```
1 instance["n_boxes"]      = pallet_info["n_boxes"]
2 instance["boxes_x_dim"]  = boxes_x_dim
3 instance["boxes_along_x"] = pallet_info["boxes_along_x"]
4 instance["boxes_along_y"] = pallet_info["boxes_along_y"]
5 instance["grip_size"]    = grip_size
6
7 result = instance.solve()
```



(a) $k = 1$



(b) $k = 2$



(c) $k = 3$

Figure 4: Solve time vs. number of boxes, split by gripper multiplier, with $S = k \cdot \text{boxes_x_dim}$. The y -axis is logarithmic.

After the solver terminates, the Python code reads the MiniZinc output arrays `group_start`, `group_len`, `drop_order`, and `plate`. These arrays describe the computed pallet layout and the corresponding placement schedule.

5 Benchmarking

The benchmark evaluates the MiniZinc model on every (pallet, pack, grip) combination defined by the benchmark grid. The pallet presets are listed in Table 1, while the pack sizes are listed in Table 2.

For each combination, the model is tested with four solvers: `gecode`, `cp-sat`, `chuffed`, and `highs`. These solvers represent and solve the problem in different ways. `gecode` and `chuffed` mainly rely on constraint propagation, `cp-sat` uses a SAT-based Boolean/integer representation, and `highs` uses a linear programming representation. Each run is executed with a timeout of 300 seconds.

Pallet preset	Dimensions (mm)
Europallet	1200×800
Industrie	1200×1000
US 40×48	1219×1016
Half-pallet	800×600
Australian	1165×1165

Table 1: Pallet presets used in the benchmark.

Pack size (mm)
50×50
70×70
100×100
120×80
150×150
200×200

Table 2: Pack sizes used in the benchmark.

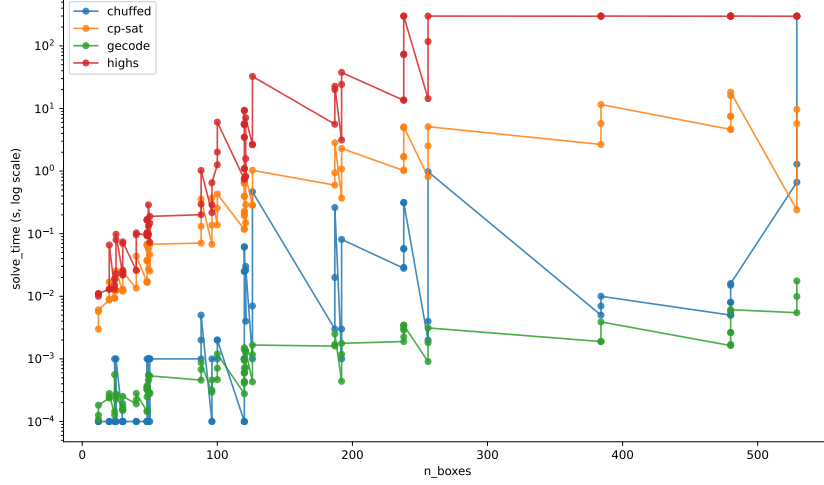


Figure 5: Solve time vs. total number of boxes — all solvers, all instances. The y -axis is logarithmic.

The gripper size is varied as an integer multiple of the box dimension along the x -axis: $S \in \{1, 2, 3\} \cdot \text{boxes_x_dim}$. This variation is used to study how the gripper size affects the feasibility and runtime of the scheduling model.

In the model, this makes `max_pkble_boxes` equal to $k + 1$. The three benchmark cases therefore correspond to maximum group lengths of two, three, and four boxes along a pallet row.

All timing plots in this section use a logarithmic scale for the time axis. This is useful because the benchmark spans several orders of magnitude: some `gecode` runs are solved in milliseconds, while several `highs` runs reach the 300-second timeout. With a linear scale, the fast solvers would be compressed near zero and the comparison would be much harder to read.

5.1 Scaling by gripper size

The first comparison keeps the gripper multiplier fixed and studies how the reported solve time changes with the total number of boxes. This separates the three different grouping regimes before aggregating them into a single plot. Figure 4 shows the resulting scaling curves for $k = 1$, $k = 2$, and $k = 3$.

The split shows the expected effect of the gripper size. Larger grippers reduce the number of groups that must be scheduled, and the average number of groups drops from about 75 for $k = 1$ to about 40 for $k = 3$. As a result, the curves generally move downward when the multiplier increases.

`gecode` stays in the lowest band and is almost flat across the whole range. `cp-sat` scales more visibly with the number of boxes, but still remains well below the timeout. `chuffed` is usually fast, with one timeout on the densest $k = 1$ instance. `highs` is consistently the slowest solver.

5.2 Overall solver behaviour

After separating the grip sizes, the full benchmark can be viewed as one aggregate scaling plot. Figure 5 shows solve time with respect to the total number of boxes on the pallet for every solver across all instances.

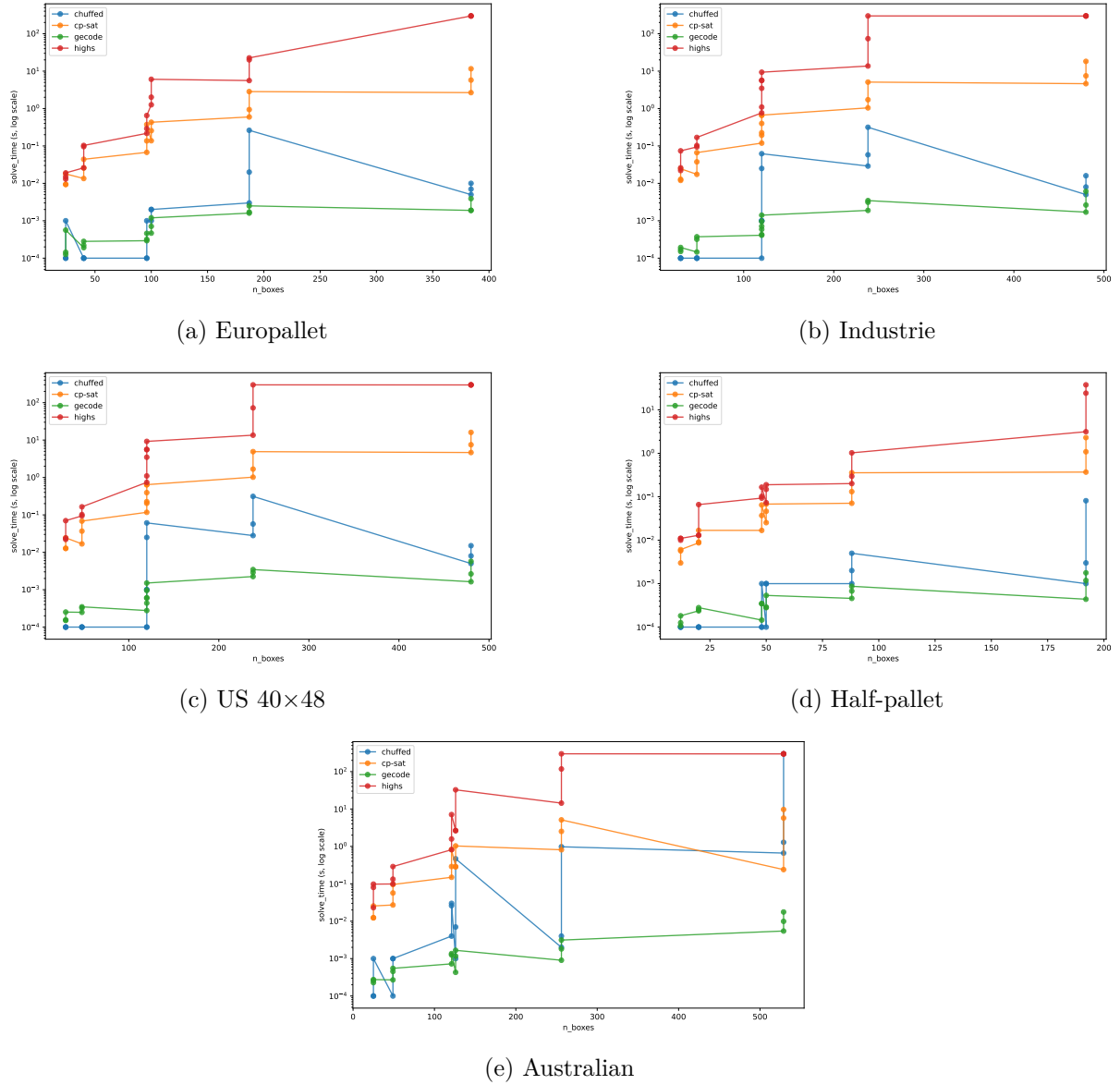


Figure 6: Solve time vs. number of boxes, broken down by pallet preset. The y -axis is logarithmic.

5.3 Per-pallet breakdown

Splitting the data by pallet preset, as shown in Figure 6, isolates the effect of pack count from pallet shape. For a fixed pallet, the curves are functions of pack count and gripper size; differences between pallets reflect the impact of `boxes_along_x` and `boxes_along_y` on the number of groups.

The larger pallets expose the same behavior seen in the aggregate plots more clearly: dense layouts with many small boxes are the hard cases, while larger boxes produce fewer groups and remain easier for all solvers. The log scale is especially useful in these plots because it shows the sub-second propagation runs and the near-timeout `highs` runs in the same figure.

5.4 Wall time vs. solve time

The gap between Python-measured wall time and solver-reported solve time captures MiniZinc flattening and inter-process overhead. Figure 7 plots the two against each other; points above the diagonal indicate runs in which the harness cost dominated the search itself. In this plot

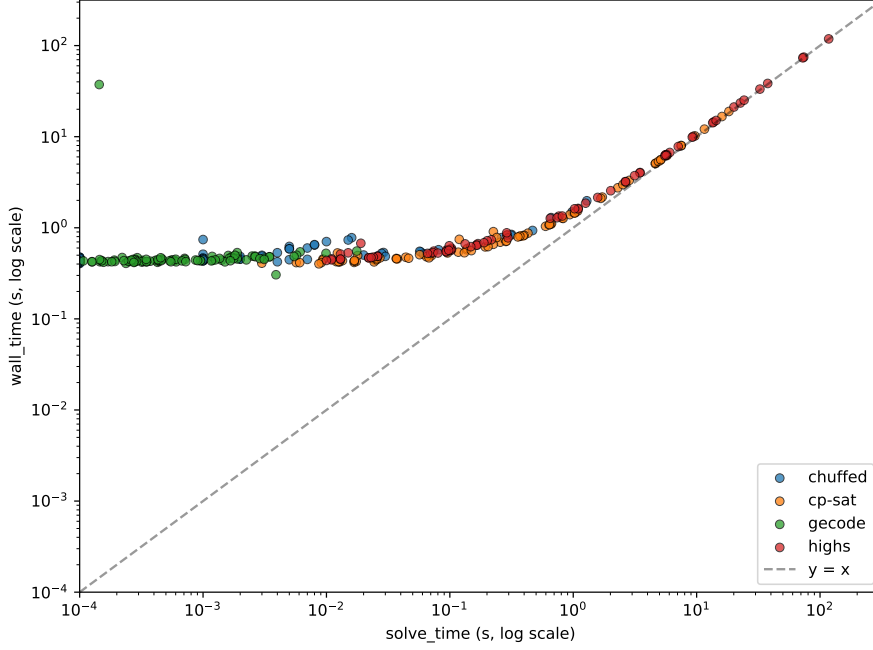


Figure 7: Wall time vs. solve time per run. The diagonal represents zero overhead; both axes are logarithmic.

both axes are logarithmic, so the small overheads of the fast runs and the long timeout-scale runs can be compared in the same view.

This distinction is especially visible for the fastest solver runs: their reported solve time is very small, so model construction, flattening, and process startup become a relevant part of the measured wall time. For the slow runs near the timeout, instead, wall time and solve time are much closer.

6 Conclusions

The project shows that the robot movement planning problem can be expressed compactly as a Constraint Programming model by separating it into grouping and scheduling decisions. Representing each group through its start index and length avoids the larger fully enumerated and binary-membership encodings, while still capturing the geometric constraints of the gripper. The release schedule can then be described with a small number of ordering variables and local neighbor constraints, which keeps the model readable and close to the physical interpretation of the task.

The benchmark confirms that the CP-oriented solvers scale well across realistic pallet sizes within the 5-minute budget. Larger grippers reduce the number of groups to schedule and generally make the instances easier, while dense layouts with many small boxes remain the most demanding cases. `gecode` is the most stable solver in these experiments, `cp-sat` and `chuffed` remain competitive, and `highs` exposes the cost of using a linear representation for this kind of combinatorial model. The comparison between wall time and solver-reported time also shows that, for very fast runs, MiniZinc flattening and process startup become a visible part of the total runtime.

Overall, the adopted formulation is a practical baseline for generating valid pick-and-place plans. Possible extensions include adding an objective function, for example to minimize placement time or favor smoother robot motions, and testing additional search annotations or symmetry-breaking constraints on the hardest dense instances.